Bytecode Table

목적

Bytecode Table은 스마트 컨트랙트의 **바이트코드를 저장하고 검증**하는 테이블입니다. 이 테이블은 Bytecode Circuit에서 생성되며, EVM Circuit에서 opcode 실행 시 바이트코드를 조회하는 데 사용됩니다. 각 바이트코드는 code hash로 식별되며, PUSH 명령어 처리를 위한 특수 필드를 포함합니다.

테이블 스키마

파일: `zkevm-circuits/src/table.rs` (line 1186)

```
pub struct BytecodeTable {
/// 행 활성화 여부
pub q_enable: Column<Fixed>,
/// 코드 해시 (Keccak256 또는 Poseidon)
pub code_hash: Column<Advice>,
/// 태그 (Header 또는 Byte)
pub tag: Column<Advice>,
/// 바이트 인덱스
pub index: Column<Advice>,
/// 코드 여부 (PUSH 데이터가 아닌 실제 opcode인지)
pub is_code: Column<Advice>,
/// 바이트 값
```

```
pub  value: Column<Advice>,

/// PUSH 데이터의 RLC (LE 순서)

pub  push_rlc: Column<Advice>,

}
```

컬럼 설명

1. q_enable (Fixed)

- 해당 행이 유효한 바이트코드 행인지 표시
- 모든 바이트코드 행에서 `1`

2. code_hash (Advice, SecondPhase)

- 바이트코드의 해시 값
- **Keccak256**: 일반 바이트코드 (EVM 표준)
- **Poseidon**: zkTrie용 바이트코드 (Scroll 확장)
- RLC로 인코딩되어 SecondPhase에 할당

3. tag (Advice)

- 행의 타입을 식별하는 태그
- `BytecodeFieldTag::Header` (0): 헤더 행
- `BytecodeFieldTag::Byte` (1): 바이트 행

4. index (Advice)

- **Header**: 바이트코드 길이
- **Byte**: 바이트 인덱스 (0부터 시작)

5. is_code (Advice)

- **Header**: 항상 `0`
- **Byte**:
- `1` : 실제 opcode (실행 가능한 코드)
- `0` : PUSH 명령어의 데이터 (실행되지 않음)

6. value (Advice)

- **Header**: 바이트코드 길이 (index와 동일)
- **Byte**: 바이트 값 (0-255)

7. push_rlc (Advice, SecondPhase)

- **Header**: `0`
- **Byte**:
- PUSH 명령어인 경우: PUSH 데이터의 RLC (Little-Endian)
- 다른 opcode인 경우: `0`
- RLC로 인코딩되어 SecondPhase에 할당

BytecodeFieldTag 열거형

파일: `zkevm-circuits/src/table.rs` (line 1173)

```
pub enum BytecodeFieldTag {
Header, // 헤더 행 (바이트코드 메타데이터)
Byte, // 바이트 행 (실제 바이트코드)
}
```

Header 행

각 바이트코드는 **하나의 Header 행**으로 시작합니다.

| 컬럼 | 값 |

|------|-----|

| `tag` | `BytecodeFieldTag::Header` (0) |

| `code_hash` | 바이트코드 해시 (RLC) |

| `index` | 바이트코드 길이 |

| `is_code` | `0` |

| `value` | 바이트코드 길이 (index와 동일) |

| `push_rlc` | `0` |

목적:

- 바이트코드의 메타데이터 제공
- 바이트코드 길이 저장
- 바이트코드 시작 표시

Byte 행

Header 행 다음에는 각 바이트에 대한 **Byte 행**이 이어집니다.

| 컬럼 | 값 |

|------|-----|

| `tag` | `BytecodeFieldTag::Byte` (1) |

| `code_hash` | 바이트코드 해시 (RLC) |

| `index` | 바이트 인덱스 (0, 1, 2, ...) |

| `is_code` | `1` (opcode) 또는 `0` (PUSH 데이터) |

| `value` | 바이트 값 (0-255) |

| `push_rlc` | PUSH 데이터 RLC 또는 `0` |

PUSH 명령어 처리

PUSH 명령어는 특별한 처리가 필요합니다. PUSH1-PUSH32는 다음 1-32바이트를 스택에 푸시하는 명령어이며, 이 데이터 바이트들은 실행되지 않습니다.

is_code 필드

`is_code` 필드는 바이트가 실제로 실행 가능한 opcode인지 구분합니다:

```
바이트코드: 60 42 60 01 01

PUSH1 0x42 PUSH1 0x01 ADD
```

```
index | value | is_code | 설명

------|-------|---------|------------------

0 | 0x60 | 1 | PUSH1 (opcode)

1 | 0x42 | 0 | PUSH1의 데이터

2 | 0x60 | 1 | PUSH1 (opcode)

3 | 0x01 | 0 | PUSH1의 데이터

4 | 0x01 | 1 | ADD (opcode)
```

push_rlc 필드

`push_rlc` 는 PUSH 명령어의 데이터를 RLC로 인코딩한 값입니다. 이는 EVM Circuit에서 PUSH 값을 효율적으로 검증하는 데 사용됩니다.

계산 방식 (Little-Endian):

```
// PUSH2 0x1234의 경우
push_data = [0x12, 0x34]
push_rlc = 0x34 + 0x12 * challenge
```

예시: `PUSH2 0x1234`

```
index | value | is_code | push_rlc
------|-------|---------|------------------
0 | 0x61 | 1 | 0 (PUSH2 opcode)
1 | 0x12 | 0 | 0x34 + 0x12 * r
2 | 0x34 | 0 | 0x34
```

여기서:

- index 0: PUSH2 opcode, `push_rlc = 0`
- index 1: 첫 번째 데이터 바이트, `push_rlc = 0x34 + 0x12 * r` (전체 PUSH 데이터)
- index 2: 두 번째 데이터 바이트, `push_rlc = 0x34` (남은 데이터)

PUSH 데이터 RLC 누적

`push_rlc` 는 **역순으로 누적**됩니다:

```
PUSH3 0xAABBCC
```

```
index | value | push_rlc 계산
```

```
------|-------|----------------------------------

0 | 0x62 | 0 (PUSH3 opcode)

1 | 0xAA | 0xCC + 0xBB * r + 0xAA * r^2

2 | 0xBB | 0xCC + 0xBB * r

3 | 0xCC | 0xCC
```

이 방식은 Bytecode Circuit에서 효율적으로 검증할 수 있도록 설계되었습니다.

바이트코드 언롤링 (Unrolling)

바이트코드는 **언롤링(unrolling)** 과정을 거쳐 Bytecode Table에 할당됩니다.

파일: `zkevm-circuits/src/bytecode_circuit/bytecode_unroller.rs`

언롤링 프로세스

1. **바이트코드 파싱**: 원시 바이트 배열을 읽음
2. **PUSH 감지**: PUSH1-PUSH32 명령어 식별
3. **is_code 설정**: 각 바이트가 opcode인지 데이터인지 표시
4. **push_rlc 계산**: PUSH 데이터의 RLC 계산
5. **테이블 행 생성**: Header 행과 Byte 행 생성

예시: 간단한 바이트코드

바이트코드: `60 42 60 01 01` (PUSH1 0x42, PUSH1 0x01, ADD)

Bytecode Table:

```
tag | code_hash | index | is_code | value | push_rlc
```

```
-------|-----------|-------|---------|-------|----------

Header | hash_rlc | 5 | 0 | 5 | 0

Byte | hash_rlc | 0 | 1 | 0x60 | 0

Byte | hash_rlc | 1 | 0 | 0x42 | 0x42

Byte | hash_rlc | 2 | 1 | 0x60 | 0

Byte | hash_rlc | 3 | 0 | 0x01 | 0x01

Byte | hash_rlc | 4 | 1 | 0x01 | 0
```

예시: PUSH32 명령어

바이트코드: `7F` + 32바이트 데이터

Bytecode Table (일부):

```
tag | code_hash | index | is_code | value | push_rlc

-------|-----------|-------|---------|-------|----------

Header | hash_rlc | 33 | 0 | 33 | 0

Byte | hash_rlc | 0 | 1 | 0x7F | 0 (PUSH32)

Byte | hash_rlc | 1 | 0 | d[0] | RLC(d[0..32])

Byte | hash_rlc | 2 | 0 | d[1] | RLC(d[1..32])

...

Byte | hash_rlc | 32 | 0 | d[31] | d[31]
```

Code Hash 검증

Bytecode Table은 바이트코드의 **무결성을 검증**하기 위해 code hash를 사용합니다.

Keccak Code Hash

일반적인 바이트코드는 Keccak256 해시를 사용합니다:

```
code_hash = keccak256(bytecode)
```

Bytecode Circuit은 Keccak Table을 조회하여 code hash를 검증합니다:

```
// Bytecode Circuit에서 Keccak Table lookup

meta.lookup_any("bytecode keccak", |meta| {

vec![

1.expr(), // q_enable

1.expr(), // is_final

bytecode_rlc.expr(), // input_rlc

bytecode_length.expr(), // input_len

code_hash_rlc.expr(), // output_rlc

]

});
```

Poseidon Code Hash

Scroll의 zkTrie는 Poseidon 해시를 사용합니다:

```
code_hash = poseidon_hash(bytecode)
```

Bytecode Circuit은 Poseidon Table을 조회하여 code hash를 검증합니다.

EVM Circuit과의 연동

EVM Circuit은 Bytecode Table을 조회하여 opcode를 가져옵니다.

Opcode Lookup

```
// EVM Circuit에서 Bytecode Table lookup
meta.lookup_any("opcode lookup", |meta| {
vec![
1.expr(), // q_enable
code_hash_rlc.expr(), // code_hash
BytecodeFieldTag::Byte.expr(), // tag
program_counter.expr(), // index
is_code.expr(), // is_code
opcode.expr(), // value
push_rlc.expr(), // push_rlc (PUSH 명령어용)
]
});
```

PUSH 값 검증

PUSH 명령어 실행 시, EVM Circuit은 `push_rlc` 를 사용하여 스택에 푸시할 값을 검증합니다:

```
// PUSH1 0x42 실행
```

```
// Bytecode Table에서 push_rlc = 0x42를 가져옴

// 스택에 0x42 푸시

stack.push(push_value);



// 제약 조건: push_value == push_rlc (PUSH1의 경우)
```

바이트코드 패딩

바이트코드가 최대 길이보다 짧은 경우, **패딩**이 추가됩니다.

패딩 규칙

1. **Header 행**: 실제 바이트코드 길이 사용
2. **Byte 행**: 실제 바이트만 포함
3. **추가 패딩 없음**: Bytecode Table은 실제 바이트만 저장

예시: 3바이트 바이트코드

```
tag | index | value

-------|-------|-------

Header | 3 | 3

Byte | 0 | 0x60

Byte | 1 | 0x42

Byte | 2 | 0x00
```

테이블 할당

Bytecode Table은 `dev_load` 메서드를 통해 할당됩니다.

파일: `zkevm-circuits/src/table.rs` (line 1227)

```
impl BytecodeTable {
pub fn dev_load<'a, F: Field>(
&self,
layouter: &mut impl Layouter<F>,
bytecodes: impl IntoIterator<Item = &'a Bytecode> + Clone,
challenges: &Challenges<Value<F>>,
) -> Result<(), Error> {
layouter.assign_region(
|| "bytecode table",
|mut region| {
let mut offset = 0;
for bytecode in bytecodes.clone() {
// Header 행 할당
// Byte 행 할당
for row in bytecode.table_assignments(challenges) {
// 각 행 할당
offset += 1;
}
}
```

```
Ok(())
},
)
}
}
```

Bytecode 구조체

파일: `zkevm-circuits/src/witness/bytecode.rs`

```
pub struct Bytecode {
pub hash: Word, // Code hash
pub bytes: Vec<u8>, // 원시 바이트
pub rows: Vec<BytecodeRow>, // 언롤링된 행
}

pub struct BytecodeRow {
pub tag: BytecodeFieldTag,
pub index: usize,
pub is_code: bool,
pub value: u8,
pub push_data_left: u64,
pub push_data_size: u64,
```

```
}
```

서브 테이블: Mini Bytecode Table

Bytecode Table은 **Mini Bytecode Table**이라는 서브 테이블을 제공합니다. 이는 `is_code` 와 `push_rlc` 를 제외한 간소화된 버전입니다.

파일: `zkevm-circuits/src/table.rs` (line 1268)

```
impl BytecodeTable {
fn columns_mini(&self) -> Vec<Column<Any>> {
vec![
self.q_enable.into(),
self.code_hash.into(),
self.tag.into(),
self.index.into(),
self.value.into(),
]
}

pub fn table_exprs_mini<F: Field>(
&self,
meta: &mut VirtualCells<F>
) -> Vec<Expression<F>> {
// Mini 테이블 표현식 반환
```

```
}

}
```

사용 사례:

- 간단한 바이트코드 조회 (PUSH 정보 불필요)
- 회로 크기 최적화

성능 최적화

1. RLC 사용

- `code_hash` : 32바이트 해시를 하나의 필드 원소로 압축
- `push_rlc` : PUSH 데이터를 효율적으로 인코딩

2. SecondPhase 컬럼

RLC를 사용하는 컬럼들은 SecondPhase에 할당:

- `code_hash`
- `push_rlc`

3. is_code 최적화

`is_code` 필드를 통해:

- PUSH 데이터와 opcode를 명확히 구분
- EVM Circuit에서 잘못된 실행 방지
- 제약 조건 간소화

제약 조건

1. Code Hash 검증

```
// Keccak Table lookup
meta.lookup_any("code hash keccak", |meta| {
vec![
1.expr(), // q_enable
1.expr(), // is_final
bytecode_rlc.expr(), // input_rlc
bytecode_length.expr(), // input_len
code_hash_rlc.expr(), // output_rlc
]
});
```

2. PUSH 데이터 검증

```
// push_rlc 누적 검증
// push_rlc[i] = value[i] + push_rlc[i+1] * challenge
```

3. is_code 일관성

```
// PUSH 명령어 다음 N바이트는 is_code = 0
// 예: PUSH2 다음 2바이트는 is_code = 0
```

디버깅 팁

Bytecode 덤프

```
// 바이트코드 출력
for (i, byte) in  bytecode.bytes.iter().enumerate() {
println!("{}: 0x{:02x} (is_code: {})",
i, byte, bytecode.rows[i].is_code);
}
```

일반적인 오류

1. **Code hash 불일치**: 바이트코드와 해시가 맞지 않음
2. **is_code 오류**: PUSH 데이터가 잘못 표시됨
3. **push_rlc 계산 오류**: RLC 누적이 잘못됨

예시: 전체 바이트코드

Solidity 코드:

```
function add(uint a, uint b) public pure returns (uint) {
return a + b;
}
```

바이트코드 (간소화):

```
60 80 60 40 52 ...

PUSH1 0x80 PUSH1 0x40 MSTORE ...
```

Bytecode Table (일부):

```
tag | code_hash | index | is_code | value | push_rlc

-------|-----------|-------|---------|-------|----------

Header | hash_rlc | 100 | 0 | 100 | 0

Byte | hash_rlc | 0 | 1 | 0x60 | 0

Byte | hash_rlc | 1 | 0 | 0x80 | 0x80

Byte | hash_rlc | 2 | 1 | 0x60 | 0

Byte | hash_rlc | 3 | 0 | 0x40 | 0x40

Byte | hash_rlc | 4 | 1 | 0x52 | 0

...
```

관련 문서

- Bytecode Circuit - Bytecode Table을 생성하는 회로
- EVM Circuit - Bytecode Table을 조회하는 회로
- Keccak Table - Code hash 검증용 테이블
- RLC 기법 - RLC 인코딩 상세

참고 자료

- **코드**: `zkevm-circuits/src/table.rs` (line 1186)
- **Witness**: `zkevm-circuits/src/witness/bytecode.rs`
- **Unroller**: `zkevm-circuits/src/bytecode_circuit/bytecode_unroller.rs`